

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

ЗВІТ
з лабораторної роботи №2

Тема: Робота з брокерами повідомлень

Варіант 1

Київ – 2025

1. Тема роботи

Робота з брокерами повідомлень. Розробка розподіленого застосунку з використанням RabbitMQ для асинхронної передачі даних між модулями.

2. Мета роботи

Вивчити принципи роботи брокерів повідомлень на прикладі RabbitMQ. Розробити двомодульний розподілений застосунок, у якому перший модуль приймає запит від клієнта і публікує повідомлення до черги, а другий модуль споживає ці повідомлення та зберігає їх у базі даних.

3. Індивідуальне завдання

Варіант 1 — Оранжерея (Flower).

Предметна область: рослини в оранжереї. Кореневий елемент — Flower.
Поля сутності:

- Name — назва рослини.
- Soil — ґрунт (підзолистий, ґрунтовий, дерново-підзолистий).
- Origin — місце походження.
- Visual parameters — зовнішні параметри: колір стебла, колір листя, середній розмір.
- Growing tips — умови зростання: температура, освітлення, полив.
- Multiplying — розмноження: листям, живцями або насінням.

4. Теоретичні відомості

RabbitMQ — брокер повідомлень, що реалізує протокол AMQP 0-9-1. Виробник (producer) публікує повідомлення до exchange, звідки вони маршрутизуються до черг (queues) згідно з binding key. Споживач (consumer) підписується на чергу і обробляє повідомлення асинхронно. Така архітектура забезпечує розв'язку між модулями та стійкість до тимчасової недоступності споживача.

Spring AMQP надає анотацію `@RabbitListener` для декларативного опису споживачів та `RabbitTemplate` для відправки. Конфігурація черги, exchange та binding оголошується через Spring beans типу Queue, DirectExchange і Binding.

5. Архітектура застосунку

Застосунок складається з двох модулів в межах одного Spring Boot проекту, що взаємодіють через RabbitMQ:

Модуль GreenProvider (виробник):

Приймає запит реєстрації квітки через gRPC-інтерфейс (FlowerRegistrationServiceImpl). Серіалізує отриманий об'єкт RegisterFlowerMessage (Protobuf) у JSON за допомогою Jackson і публікує до черги RabbitMQ через RabbitTemplate.

Модуль Greenhouse (споживач):

Підписується на чергу через @RabbitListener (FlowerClientService). Десеріалізує JSON-повідомлення у сутність FlowerEntity та зберігає її у реляційній базі даних через Spring Data JPA. Для розробки і тестування використовується вбудована H2-база.

6. Хід виконання роботи

6.1. Proto-контракт (flower.proto)

```
message RegisterFlowerMessage {
    string name          = 1;
    FlowerSoilType soil = 2;
    string origin        = 3;
    VisualParameters visual = 4;
    repeated GrowingTip tips = 5;
    string multiplying = 6;
}

enum FlowerSoilType {
    PODZOLIC = 0;
    GROUND   = 1;
    SOD_PODZOLIC = 2;
}
```

6.2. Конфігурація RabbitMQ

Клас RabbitConfiguration оголошує чергу, direct exchange і binding із routing key 'flower.register'. Брокер запускається локально через Docker (образ rabbitmq:3-management) командою з compose.yaml.

```
docker compose up -d # піднімає RabbitMQ на localhost:5672
```

6.3. Збереження у базі даних

FlowerEntity — JPA-сутність з полями, що відповідають proto-структурі. Вбудовані об'єкти VisualParameters і GrowingTip зберігаються як @Embeddable.

FlowerRepository розширює JpaRepository, надаючи стандартні CRUD-операції без додаткового коду.

6.4. Перевірка роботи

Після запуску застосунку та RabbitMQ надіслано gRPC-запит через `grpcurl` із заповненими полями Flower. У логах GreenProvider зафіксовано публікацію до черги; у логах Greenhouse — отримання та успішне збереження записи. Наявність запису підтверджено через H2 Console (<http://localhost:8080/h2-console>).

7. Висновки

Розроблено розподілений застосунок на основі gRPC і RabbitMQ у рамках предметної області «Оранжерея». Реалізовано повний цикл обробки даних: прийом запиту через gRPC → серіалізація у JSON → публікація до черги RabbitMQ → асинхронне споживання → збереження у базі даних. Засвоєно принципи AMQP-маршрутизації, конфігурації Spring AMQP та інтеграції двох протоколів (gRPC і AMQP) в єдиному застосунку.

8. Посилання

Репозиторій з вихідним кодом:
<https://github.com/javaAndScriptDeveloper/lolitech/tree/main/defi/submission2>